



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC1001 — Algoritmos y Sistemas Computacionales — 2026-1 Actividad Evaluada 2 - Operaciones numéricas y codificación

Jueves 26-Marzo-2026

Instrucciones

Deberán realizar esta actividad dentro de sus grupos designados según Canvas.

Entrega: Jueves 26-Marzo, 12:10 mediante foto en Canvas

- Pueden usar apuntes durante la actividad.
- Pueden hacer consultas al equipo docente durante la actividad.
- Deben resolver manualmente, sin ayuda de computador u otros dispositivos electrónicos.
- No se realiza en computador.
- Cualquier miembro puede hacer la entrega.

Parte 1. Suma de Binarios [4pts]

¡DCCin está aportando a la construcción del 5to piso! para ello le pidieron calcular el perímetro de algunas salas, pero el problema es que DCCin solo entiende números binarios :(... y aún es algo torpe con las sumas.

Como traductor oficial del DCC, deberás ayudarlo a realizar la operación de sumar las dimensiones y luego entregarle a los arquitectos el resultado en decimal.

Los operaciones que te da DCCin son las siguientes (en algunas estaba tan cansado que para escribir menos lo hizo en hexadecimal).

(a) $01000000_2 + 00000011_2$

R.

$$\begin{array}{r} 01000000 \\ + 00000011 \\ \hline 01000011 \end{array}$$

01000011_2 corresponde a $1 \times 2^6 + 1 \times 2^1 + 1 \times 2_0 = 64 + 2 + 1 = 67_{10}$. A modo de comprobación podemos ver que la suma en decimal corresponde a $64_{10} + 3_{10} = 67_{10}$

(b) $00011111_2 + 00001111_2$

R.

$$\begin{array}{r}
 11111 \\
 00011111 \\
 + 00001111 \\
 \hline
 00101110
 \end{array}$$

00101110_2 corresponde a $1 \times 2^5 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 = 32 + 8 + 4 + 2 = 46_{10}$. A modo de comprobación podemos ver que la suma en decimal corresponde a $31_{10} + 15_{10} = 46_{10}$

(c) $AA_{16} + 0F_{16}$

R.

Primero expresamos cada dígito hexadecimal como 4 bits:

$$AA_{16} + 0F_{16} = 10101010_2 + 00001111_2$$

$$\begin{array}{r}
 111 \\
 10101010 \\
 + 00001111 \\
 \hline
 10111001
 \end{array}$$

10111001_2 corresponde a $1 \times 2^7 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^0 = 128 + 32 + 16 + 8 + 1 = 185_{10}$. A modo de comprobación podemos ver que la suma en decimal corresponde a $170_{10} + 15_{10} = 185_{10}$

(d) $81_{16} + 14_{16}$

Primero expresamos cada dígito hexadecimal como 4 bits:

$$81_{16} + 14_{16} = 10000001_2 + 00010100_2$$

$$\begin{array}{r}
 11111 \\
 10000001 \\
 + 00010100 \\
 \hline
 10010101
 \end{array}$$

10010101 corresponde a $1 \times 2^7 + 1 \times 2^4 + 1 \times 2^2 + 1 \times 2^0 = 128 + 16 + 4 + 1 = 149_{10}$. A modo de comprobación podemos ver que la suma en decimal corresponde a $129_{10} + 20_{10} = 149_{10}$

Parte 2. La negatividad de DCCin [4pts]

DCCin ve el mundo de manera demasiado positiva, literalmente, ya que cuando ve números solo ve positivos, eso ha alterado el cálculo de dimensiones anterior. Para ello deberás darle un par de ejemplos en números negativos, para que se vaya familiarizando...

Debes expresar los siguientes números negativos en binario utilizando el formato complemento de dos en 8 bits (*signed 8-bit*).

Estos números están escritos en base 10.

(a) -49

R.

Usando 8 bits en complemento a dos, 49_{10} corresponde a $32 + 16 + 1 = 2^5 + 2^4 + 2^0 = 00110001_2$. Para obtener la versión negativa, invertimos los bit y sumamos uno: $00110001 \rightarrow 11001110 \rightarrow \mathbf{11001111}$.

(b) -67

R.

Usando 8 bits en complemento a dos, 67_{10} corresponde a $64 + 2 + 1 = 2^6 + 2^1 + 2^0 = 01000011_2$. Para obtener la versión negativa, invertimos los bit y sumamos uno: $01000011 \rightarrow 10111100 \rightarrow \mathbf{10111101}$.

(c) -99

R.

Usando 8 bits en complemento a dos, 99_{10} corresponde a $64 + 32 + 2 + 1 = 2^6 + 2^5 + 2^1 + 2^0 = 01100011_2$. Para obtener la versión negativa, invertimos los bit y sumamos uno: $01100011 \rightarrow 10011100 \rightarrow \mathbf{10011101}$.

(d) -81

R.

Usando 8 bits en complemento a dos, 81_{10} corresponde a $64 + 16 + 1 = 2^6 + 2^4 + 2^0 = 01010001_2$. Para obtener la versión negativa, invertimos los bit y sumamos uno: $01010001 \rightarrow 10101110 \rightarrow \mathbf{10101111}$.

(e) -77

R.

Usando 8 bits en complemento a dos, 77_{10} corresponde a $64 + 8 + 4 + 1 = 2^6 + 2^3 + 2^2 + 2^0 = 01001101_2$. Para obtener la versión negativa, invertimos los bit y sumamos uno: $01001101 \rightarrow 10110010 \rightarrow \mathbf{10110011}$.

(f) -111

R.

Usando 8 bits en complemento a dos, 111_{10} corresponde a $64 + 32 + 8 + 4 + 2 + 1 = 2^6 + 2^5 + 2^3 + 2^2 + 2^1 + 2^0 = 01101111_2$. Para obtener la versión negativa, invertimos los bit y sumamos uno: $01101111 \rightarrow 10010000 \rightarrow \mathbf{10010001}$.

(g) -11

R.

Usando 8 bits en complemento a dos, 11_{10} corresponde a $8 + 2 + 1 = 2^3 + 2^1 + 2^0 = 00001011_2$. Para obtener la versión negativa, invertimos los bit y sumamos uno: $00001011 \rightarrow 11110100 \rightarrow \mathbf{11110101}$.

(h) -128

R.

Usando 8 bits, 128_{10} corresponde a $128 = 2^7 = 10000000_2$. Para obtener la versión negativa, invertimos los bit y sumamos uno: $10000000 \rightarrow 01111111 \rightarrow \mathbf{10000000}$.

Este caso es límite porque usando 8 bit en complemento a dos (8 bit *signed*), no se puede representar el 128. Con 8 bit *signed* se puede representar desde el $-128_{10} = 10000000_2$ hasta el $127_{10} = 01111111_2$. La conversión que hicimos desde 10000000_2 nos llevó a la misma secuencia 10000000_2 .

Parte 3. Sustrayendo con DCCin [4pts]

Ahora pondrás a prueba los conocimientos de DCCin sobre operaciones matemáticas, verás que tan bueno es restando, pero para ello necesitarás una pauta de los resultados en **complemento a dos** en 8 bit. Realiza las siguientes operaciones matemáticas y expresa el resultado en binario complemento a dos de 8 bit.

(a) $00100010_2 - 00100011_2$

R.

Para obtener la respuesta necesitamos la versión negativa del byte 00100011_2 . Usando 8 bit en complemento a dos, esto corresponde a $00100011 \rightarrow 11011100 \rightarrow 11011101$.

El siguiente paso es calcular la suma $00100010_2 + 11011101_2$:

$$\begin{array}{r} 00100010 \\ + 11011101 \\ \hline 11111111 \end{array}$$

El resultado es 11111111_2 que es un número negativo (bit más significativo es 1). Para encontrar la versión positiva, hacemos la conversión: $11111111 \rightarrow 00000000 \rightarrow 00000001$ que corresponde a 1_{10} .

Por lo tanto, el resultado de la resta es -1_{10} . A modo de comprobación podemos ver que la operación en decimal es: $18 - 19 = -1$.

(b) $01000100_2 - 00000001_2$

R.

Para obtener la respuesta necesitamos la versión negativa del byte 00000001_2 . Usando 8 bit en complemento a dos, esto corresponde a $00000001 \rightarrow 11111110 \rightarrow 11111111$.

El siguiente paso es calcular la suma $01000100_2 + 11111111_2$:

$$\begin{array}{r} 111111 \\ 01000100 \\ + 11111111 \\ \hline 101000011 \end{array}$$

El resultado da un valor de 9 bit. Como estamos usando 8 bit, el noveno bit se descarta y queda como *carry out*. El resultado es 01000011_2 que es un número positivo y corresponde a $64 + 2 + 1 = 67_{10}$

A modo de comprobación podemos ver que la operación en decimal es: $68 - 1 = 67$.

(c) $03_{16} - 46_{16}$

R.

Ahora nos entregan los byte en notación hexadecimal. Antes de hacer la operación podemos convertirlo a binario:

$$00000011_2 - 01000110_2$$

Para obtener la respuesta necesitamos la versión negativa del byte $46_{16} = 01000110_2$. Usando 8 bit en complemento a dos, esto corresponde a $01000110 \rightarrow 10111001 \rightarrow 10111010$ (que también corresponde al byte BA_{16}).

El siguiente paso es calcular la suma $00000011_2 + 10111010_2$:

$$\begin{array}{r} 1 \\ 00000011 \\ + 10111010 \\ \hline 10111101 \end{array}$$

El resultado es 10111101_2 que es un número negativo. Para identificar la magnitud, hacemos la conversión $10111101 \rightarrow 01000010 \rightarrow 01000011$ y corresponde a $64 + 2 + 1 = 67_{10}$. Por lo tanto, el resultado de la resta es -67_{10} .

A modo de comprobación podemos ver que la operación en decimal es: $3 - 70 = -67$.

(d) $7C_{16} - 01_{16}$

R.

Nos entregan los byte en notación hexadecimal. Antes de hacer la operación podemos convertirlo a binario:

$$01111100_2 - 00000001_2$$

Para obtener la respuesta necesitamos la versión negativa del byte $01_{16} = 00000001_2$. Usando 8 bit en complemento a dos, esto corresponde a $00000001 \rightarrow 11111110 \rightarrow 11111111$ (que también corresponde al byte FF_{16}).

El siguiente paso es calcular la suma $01111100_2 + 11111111_2$:

$$\begin{array}{r} 111111 \\ 01111100 \\ + 11111111 \\ \hline 101111011 \end{array}$$

El resultado da un valor de 9 bit. Como estamos usando 8 bit, el noveno bit se descarta y queda como *carry out*. El resultado es 01111011_2 que es un número positivo, y corresponde a $64 + 32 + 16 + 8 + 2 + 1 = 123$. Por lo tanto, el resultado de la resta es 123_{10} .

A modo de comprobación podemos ver que la operación en decimal es: $124 - 1 = 123$.

Parte 4. Descifrar el mensaje ASCII [4pts]

Ya le entregaste todos los datos necesarios a DCCin, y ahora que DCCin ya está más familiarizado con los números y las salas sólo queda traducir su respuesta para comunicársela a los arquitectos. De esta manera la construcción del quinto piso del DCC podrá empezar.

Debes decodificar solo el mensaje correspondiente a tu número de grupo utilizando la codificación ASCII.

Grupo 1	0x24 0x61 0x6C 0x34 0x5F 0x64 0x65 0x5F 0x33 0x73 0x74 0x75 0x64 0x69 0x30 0x2D 0x30 0x31
Grupo 2	0x4F 0x66 0x31 0x63 0x69 0x6E 0x34 0x5F 0x64 0x65 0x5F 0x43 0x52 0x75 0x7A 0x2D 0x30 0x31
Grupo 3	0x3C 0x4F 0x66 0x69 0x63 0x31 0x6E 0x61 0x5F 0x59 0x61 0x64 0x72 0x34 0x6E 0x5F 0x45 0x3E
Grupo 4	0x4F 0x66 0x31 0x63 0x69 0x6E 0x34 0x5F 0x43 0x61 0x62 0x72 0x34 0x5F 0x43 0x2D 0x30 0x31
Grupo 5	0x4F 0x66 0x69 0x63 0x31 0x6E 0x61 0x5F 0x47 0x61 0x62 0x31 0x5F 0x53 0x2D 0x30 0x30 0x31
Grupo 6	0x28 0x4F 0x66 0x31 0x63 0x69 0x6E 0x61 0x5F 0x41 0x6C 0x33 0x54 0x34 0x70 0x69 0x61 0x29
Grupo 7	0x4F 0x66 0x69 0x63 0x31 0x6E 0x61 0x5F 0x56 0x61 0x6C 0x65 0x72 0x31 0x34 0x2D 0x30 0x31
Grupo 8	0x42 0x61 0x6E 0x7E 0x6F 0x5F 0x55 0x6E 0x69 0x76 0x33 0x72 0x73 0x34 0x6C 0x2D 0x30 0x31
Grupo 9	0x3C 0x45 0x73 0x63 0x34 0x6C 0x65 0x72 0x61 0x5F 0x43 0x61 0x72 0x34 0x63 0x30 0x6C 0x3E
Grupo 10	0x3C 0x41 0x73 0x63 0x33 0x6E 0x73 0x30 0x72 0x5F 0x53 0x75 0x72 0x5F 0x44 0x43 0x43 0x3E
Grupo 11	0x34 0x73 0x63 0x33 0x6E 0x73 0x6F 0x72 0x5F 0x4E 0x6F 0x72 0x74 0x65 0x5F 0x44 0x43 0x43
Grupo 12	0x52 0x65 0x63 0x65 0x70 0x63 0x31 0x6F 0x6E 0x5F 0x4E 0x30 0x72 0x74 0x65 0x2D 0x30 0x31
Grupo 13	0x52 0x65 0x63 0x33 0x70 0x63 0x69 0x6F 0x6E 0x5F 0x53 0x75 0x72 0x5F 0x44 0x43 0x43 0x2E
Grupo 14	0x43 0x6F 0x63 0x69 0x6E 0x34 0x46 0x75 0x6E 0x63 0x31 0x6F 0x6E 0x61 0x72 0x69 0x30 0x73
Grupo 15	0x53 0x61 0x6C 0x34 0x5F 0x64 0x65 0x5F 0x45 0x73 0x74 0x75 0x64 0x31 0x6F 0x2D 0x30 0x32
Grupo 16	0x53 0x34 0x6C 0x61 0x5F 0x64 0x65 0x5F 0x33 0x73 0x74 0x75 0x64 0x69 0x6F 0x2D 0x30 0x33
Grupo 17	0x4F 0x66 0x69 0x63 0x69 0x6E 0x34 0x5F 0x64 0x33 0x5F 0x43 0x52 0x75 0x7A 0x2D 0x30 0x32
Grupo 18	0x5B 0x4F 0x66 0x69 0x63 0x31 0x6E 0x61 0x5F 0x59 0x34 0x64 0x72 0x61 0x6E 0x5F 0x45 0x5D
Grupo 19	0x4F 0x66 0x31 0x63 0x69 0x6E 0x61 0x5F 0x43 0x61 0x62 0x72 0x61 0x5F 0x43 0x2D 0x30 0x32
Grupo 20	0x4F 0x66 0x69 0x63 0x69 0x6E 0x34 0x5F 0x47 0x34 0x62 0x31 0x5F 0x53 0x2D 0x30 0x30 0x32
Grupo 21	0x28 0x30 0x66 0x69 0x63 0x69 0x6E 0x61 0x5F 0x41 0x6C 0x33 0x54 0x61 0x70 0x31 0x34 0x29
Grupo 22	0x4F 0x66 0x31 0x63 0x69 0x6E 0x61 0x5F 0x56 0x61 0x6C 0x65 0x72 0x69 0x61 0x2D 0x30 0x32
Grupo 23	0x42 0x61 0x6E 0x7E 0x6F 0x5F 0x55 0x6E 0x69 0x76 0x65 0x72 0x73 0x61 0x6C 0x2D 0x30 0x32
Grupo 24	0x24 0x61 0x6C 0x34 0x5F 0x64 0x65 0x5F 0x45 0x73 0x74 0x75 0x64 0x69 0x6F 0x2D 0x30 0x34
Grupo 25	0x53 0x34 0x6C 0x61 0x5F 0x64 0x65 0x5F 0x52 0x33 0x75 0x6E 0x69 0x6F 0x6E 0x33 0x73 0x2E
Grupo 26	0x4F 0x66 0x31 0x63 0x31 0x6E 0x61 0x5F 0x64 0x65 0x5F 0x43 0x52 0x75 0x7A 0x2D 0x30 0x33
Grupo 27	0x3C 0x4F 0x66 0x69 0x63 0x31 0x6E 0x61 0x5F 0x59 0x34 0x64 0x72 0x34 0x6E 0x5F 0x45 0x3E
Grupo 28	0x4F 0x66 0x69 0x63 0x69 0x6E 0x61 0x5F 0x4A 0x30 0x72 0x67 0x65 0x50 0x33 0x72 0x65 0x7A
Grupo 29	0x3C 0x4F 0x66 0x69 0x63 0x69 0x6E 0x61 0x5F 0x53 0x75 0x73 0x34 0x6E 0x61 0x5F 0x46 0x3E
Grupo 30	0x4F 0x66 0x31 0x63 0x69 0x6E 0x61 0x5F 0x53 0x65 0x62 0x34 0x42 0x75 0x67 0x33 0x64 0x30
Grupo 31	0x30 0x66 0x69 0x63 0x31 0x6E 0x61 0x5F 0x56 0x34 0x6C 0x33 0x72 0x69 0x61 0x2D 0x30 0x33

R.

Usando la tabla de codificación ASCII, los mensajes respectivos son los siguientes.

Grupo 1	\$al4_de_3studi0-01
Grupo 2	Of1cin4_de_CRuz-01
Grupo 3	<Oficl1na_Yadr4n_E>
Grupo 4	Of1cin4_Cabr4_C-01
Grupo 5	Oficl1na_Gabl_S-001
Grupo 6	(Of1cina_Al3T4pia)
Grupo 7	Oficl1na_Valer14-01
Grupo 8	Ban~o_Univ3rs4l-01
Grupo 9	<Esc4lera_Car4c0l>
Grupo 10	<Asc3ns0r_Sur_DCC>
Grupo 11	4sc3nsor_Norte_DCC
Grupo 12	Recepclon_N0rte-01
Grupo 13	Rec3pcion_Sur_DCC.
Grupo 14	Cocin4Funcionari0s
Grupo 15	Sal4_de_Estud1o-02
Grupo 16	S4la_de_3studio-03
Grupo 17	Oficin4_d3_CRuz-02
Grupo 18	[Oficl1na_Y4dran_E]
Grupo 19	Of1cina_Cabra_C-02
Grupo 20	Oficin4_G4bl_S-002
Grupo 21	(Oficina_Al3Tap14)
Grupo 22	Of1cina_Valeria-02
Grupo 23	Ban~o_Universal-02
Grupo 24	\$al4_de_Estudio-04
Grupo 25	S4la_de_R3union3s.
Grupo 26	Of1cl1na_de_CRuz-03
Grupo 27	<Oficl1na_Y4dr4n_E>
Grupo 28	Oficina_J0rgeP3rez
Grupo 29	<Oficina_Sus4na_F>
Grupo 30	Of1cina_Seb4Bug3d0
Grupo 31	Oficl1na_V4l3ria-03